

OmniSearch

Project Documentation

Heterogeneous air-ground robotic swarms for wildfire survivor search. MIDS Capstone · Summer 2026 · UC Berkeley · Team: Ann-Kathrin Schütz · Oleksii Lavrenin · Jefferson-Stanley Jules

A single consolidated reference covering the three technical pillars of the project — **Computer Vision, Reinforcement Learning (MARL)**, and the **Datasets / data sources** — plus the simulation model that ties them together. Each section links to the deeper standalone docs and the code that implements it.

When wildfires trap people and every second counts, our AI-trained drone and ground robot teams find survivors that human rescuers can't reach in time.

Table of contents

1. Overview & research question
 2. Architecture & stack
 3. Computer Vision — survivor detection
 4. Reinforcement Learning — MARL coordination
 5. Simulation model
 6. Datasets & data sources (links)
 7. Mission metrics
 8. Repository map
 9. Source documents
-

1. Overview & research question

Wildfires routinely strand people in hazardous, hard-to-reach locations. Ground search-and-rescue teams cannot safely enter active burn zones; aerial drones can survey large areas but cannot verify survivors at close range. **No existing product integrates aerial drones and ground robots into an autonomous coordinated search-and-rescue system.** OmniSearch is the *coordination layer* that connects aerial eyes to ground hands.

Research question:

*Can heterogeneous air-ground robot teams learn cooperative survivor verification strategies that **outperform hand-coded heuristics**, and **degrade gracefully under communication dropout**?*

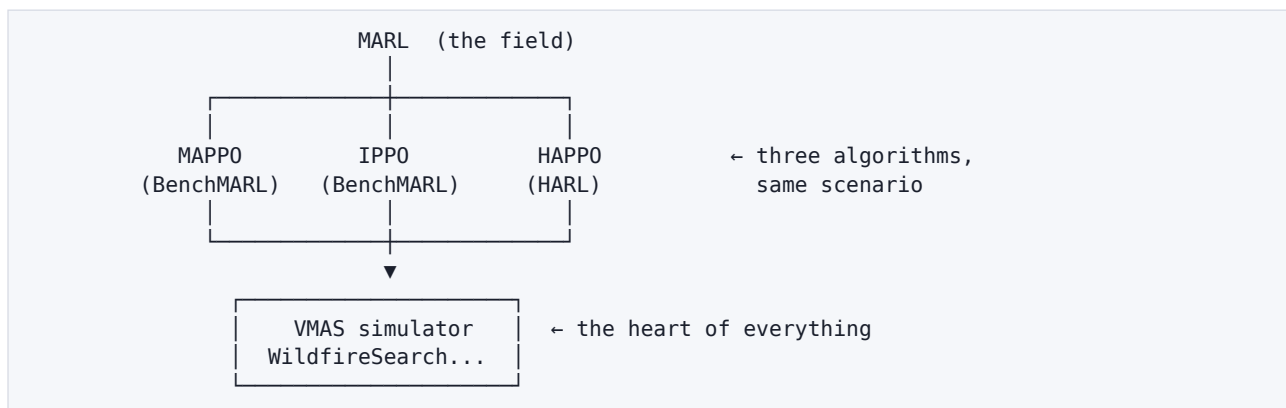
Three sub-questions answered in simulation:

1. **Heterogeneity** — do drones + ground robots beat drones-only / ground-only?

2. **Coordination** — does learned MARL beat hand-coded baselines (nearest-candidate, highest-confidence, lawnmower)?
3. **Robustness** — how does each strategy degrade across 0% → 70% comms dropout?

2. Architecture & stack

MARL is the field. **MAPPO**, **IPPO**, and **HAPPO** are three algorithms inside it. **VMAS** is the simulator they all train inside. All three algorithms train the same **WildfireSearchScenario**, so their results are directly comparable on the same mission metrics.



In one sentence: **VMAS = where things happen. MAPPO / IPPO / HAPPO = how policies are trained. MARL = the field all three belong to.**

Layer	Tool	Notes
Multi-agent sim	VMAS	2D, CPU-vectorized, fast
Fire spread	Cellular automata over a 128×128 grid	SimFire compatible (needs Python 3.9–3.10)
MARL training	BenchMARL (MAPPO, IPPO) + HARL (HAPPO)	All three train the same scenario
Detection	Ultralytics YOLOv8 with <code>classes=[0]</code>	Person = COCO class 0
Experiment tracking	Weights & Biases	Optional; pass <code>loggers=["wandb"]</code>
Deliverable	React + Three.js viewer	Strategy comparison & replay

HAPPO is wired. *BenchMARL 1.x ships MAPPO/IPPO/MADDPG/MASAC/QMIX/VDN/IQL — not HAPPO. True HAPPO (Kuba 2022) lives in [HARL](#). We bridge HARL to our VMAS scenario via `agents/harl_env.py` and run it via `scripts/train_happo_smoke.py`.*

3. Computer Vision — survivor detection

Deep dive: [docs/CV_SURVIVOR_DETECTION.md](#)

How OmniSearch detects survivors in rendered drone imagery, why the naive approach failed, and what was done to fix it.

3.1 Pipeline

Each drone frame is rendered and (optionally) run through a person detector in `detection/simulation_adapter.py`:

```
NAIP aerial background crop (real imagery, per terrain bbox)
→ render burn scar + flames (detection/wildfire_effects.py)
→ paste SARD survivor cutout(s) (real people, drone-view)
→ render smoke over the scene
→ detect people → PreliminaryPersonDetector OR YOLOv8
```

The detection backend is selectable:

- `preliminary` (default) — echoes renderer ground-truth boxes with optional noise. Fast, deterministic, no model. For wiring/tests.
- `yolo` — real YOLOv8 person detection over the rendered crop.

3.2 The problem the naive approach hit

1. **Detection bypassed the image.** The pipeline only ever used the ground-truth stub — no actual computer vision ran on the rendered crop.
2. **Survivors are tiny.** At realistic drone altitudes (20–50 m), a person is only ~20–48 px in a 512 px crop — below YOLO's reliable detection floor. Stock YOLOv8 detected ~0 survivors at that size.
3. **Stock YOLO is out-of-distribution.** COCO weights are trained on eye-level photos; top-down aerial survivors detect at noisy 0.2–0.7 confidence.

3.3 What was done

1. **Real YOLO backend** wired into the adapter (`--cv-detector yolo`).
2. **Tiled small-object inference** — the frame is sliced into an overlapping grid, each tile run at high `imgsz`, boxes mapped back and de-duplicated with NMS. Makes ~48 px survivors detectable where single-pass found none.
3. **IoU matching to ground truth** so each detection is tied to a survivor index (unmatched boxes count as false positives).
4. **Fine-tuning on the render distribution** (`scripts/train_survivor_detector.py`) — labelled composites of SARD survivors on terrain-like backgrounds with wildfire effects.

3.4 Two-stage detection: drone scouts, ground robot confirms

Stage	Agent	View	Why it works
Scout	Drone	Top-down aerial, survivor ~20–100 px (tiny)	Tiled high- <code>imgsz</code> inference + fine-tuned model
Confirm	Ground robot	Close-range, survivor 25–40% of frame (large)	Single-pass inference; survivor is already big

Key subtlety: tiling helps the drone's tiny survivors but *hurts* the ground robot's large ones (0.29 tiled vs 0.70–0.73 single-pass). So `render_ground_confirmation` disables tiling for the close-range view. **One yolov8s model serves both agents** — capacity beat size-specialization.

3.5 The procedural-background trap (and the NAIP fix)

A model fine-tuned on **procedural** (synthetic noise) backgrounds looked great on synthetic eval (~0.84) but **failed on the real pipeline** — it fired ~23 false positives per empty NAIP crop because it never learned what real terrain looks like. The fix: train composites on **real NAIP crops**.

Model	Real survivor recall	False positives / empty crop
Procedural-trained	1.00	23.0
NAIP-trained @ conf 0.40	0.95	1.15

Lesson: always train the detector on the same imagery it will be deployed on.
The adapter auto-prefers `models/survivor_naip_yolov8s.pt` (default conf 0.35).

3.6 Domain-gap limitations (honest)

- **No real people-in-fire imagery exists** — "a survivor in fire" is a real drone-view person with a *synthetic* flame/smoke layer over them.
- **Only one ~1 km NAIP area (4 tiles)** was available, so the FP eval is partly in-distribution. Next step: NAIP training data spanning many regions.
- **Visible-light only.** Real SAR drones use thermal/IR to see through smoke; a thermal channel would be the largest robustness gain but needs data we lack.
- Confidence numbers reflect our **simulated** fire, not field conditions.

3.7 CV code & usage

Code: `detection/fire_detector.py` (HSV + connected components),
`detection/person_detector.py` (YOLOv8 wrapper), `detection/pipeline.py` (two-stage),
`detection/naip.py`, `detection/wildfire_effects.py`, `detection/simulation_adapter.py`.

```
# Fine-tune the survivor detector on the render distribution (CPU-friendly):
python scripts/train_survivor_detector.py --epochs 30 --n-train 400 --fire-frac 0.7

# Export trajectories with real CV detection:
python scripts/export_trajectories.py --enable-cv --cv-detector yolo \
  --terrain-cache-path data/terrain_cache/<cache>.npz --cv-image-size 1024

# Visual + numeric check under fire and smoke (uses the NAIP-trained model):
python scripts/verify_survivor_cv.py --model models/survivor_naip_yolov8s.pt
```

4. Reinforcement Learning — MARL coordination

Deep dive: `docs/RL_TRAINING_EXPERIMENTS.md` (+ `.pdf`)

Question: can a trained MARL policy (HAPPO) match or beat hand-coded coordination heuristics on **survivor recall** (fraction of 5 survivors confirmed by a ground robot)?
Drones scout (broad downward camera); ground robots confirm (precise, slow).

4.1 Evaluation conditions

- **Terrain:** real cached terrain, ~1 km bbox, 128×128 grid.
- **Episode:** 1000 steps; `recall = found / 5` at episode end; mean over 5–10 seeds.
- **Reward-signal floors (opt-in, default 0.0):** `drone_min_footprint = 0.15`,
`ground_confirm_min = 0.20`. On real-km terrain the *physical* footprint shrinks too far to

give a usable reward; the floors restore it. **Both HAPPO and the baselines are evaluated at the same floors** — apples-to-apples.

4.2 Non-learning baselines (the bar to beat)

Baseline	How it works	scouted	recall
random	everyone moves randomly (control)	0.62	0.06
nearest_candidate	random drones; UGVs → nearest scouted survivor	0.60	0.62
lawnmower	serpentine drone sweep; UGVs A* to nearest scouted	0.82	0.92

4.3 HAPPO training experiments (chronological)

Each run diagnosed and fixed a distinct bottleneck. * = short diagnostic run.

#	Episodes	hrs	Key config uration	scouted	recall	Finding / bottleneck
1	100	0.1	default 22 km terrain, no floors	0.2	0.07	Wrong terrain + no reward signal
2	1000	0.6	1 km, footprint floor 0.15	0.33	0.00	Cost-dominated reward → do-nothing optimum
3	2000	1.1	+entropy 0.08, search-reward	0.16	0.00	Reward-hack s dense shaping; action saturation
4	300*	0.2	+std_y 1.0, +entropy 0.25	0.47	0.00	Action saturation broken; scouting recovers
5	3000	2.0	std1.0 + ent0.25 + search	0.56	0.00	Scouting up; ground robots never confirm
6	4000	2.6	+ground_co nfirm floor 0.12	0.36	0.16	Recall lifts off 0 for the first time
7	10000	8.6	same (non-r ecurrent)	0.33	0.10	Plateau — more budget does NOT help
8	6000	5.2	+coverage reward +recurrent (GRU)	0.62	0.12	Coverage br eakthrough

#	Episodes	hrs	Key configuration	scouted	recall	Finding / bottleneck
9	300*	0.2	+ground approach reward, confirm 0.20	0.56	0.20	Ground-confirm leg lifts recall to 0.20
10	6000	5.9	all fixes (coverage+recurrent+approach)	0.56	0.20	Plateaus at 0.20

Total HAPPO training ≈ 31 hours. On CPU (Apple M-series): ~150–280 env-steps/sec; recurrent (GRU) runs ~30% slower.

4.4 Learning from demonstration (cloning the lawnmower expert)

Pure RL plateaus at recall ~0.20. To approach the heuristic, clone its behaviour, then RL-fine-tune.

Approach	scouted	recall	Note
Behaviour cloning (feedforward), no RL	0.74	0.30	Clones the sweep; ground-confirm precision doesn't transfer
BC + RL fine-tune	0.46	0.36	RL erodes the cloned sweep
Recurrent BC (GRU, BPTT)	0.74	0.38	Best learned — memory captures multi-step navigation
Dagger (clone precision, 8 iters)	0.56	0.34	Improves over feedforward BC; below recurrent BC

4.5 Comparison: learned vs. non-learning

Policy	Trained?	recall
HAPPO — pure RL (best, all fixes)	yes	0.20
HAPPO — BC + RL fine-tune	yes	0.36
HAPPO — recurrent BC (best learned)	yes	0.38
random	no	0.06
nearest_candidate	no	0.62
lawnmower	no	0.92

4.6 Conclusion (honest)

- HAPPO improved from a do-nothing collapse (0.00) to a functioning searcher — best learned recall **0.38** (recurrent BC). Drone scouting became competitive with the baselines.
- **It does NOT beat the hand-coded baselines** (nearest 0.62, lawnmower 0.92). Pure RL plateaus at ~0.20; more episodes do not help.

- The residual gap is the ground robots' A* approach-to-survivor **precision**, which neither RL nor feedforward BC reproduces.
- **Scientific takeaway:** hand-coded coordination heuristics remain superior on mission recall; learned MARL matches the individual *scouting* sub-behaviour but not the full *scout* → *confirm* coordination at this budget.

4.7 RL code & usage

Code: [agents/harl_env.py](#) (HARL-shape adapter), [agents/harl_vec_env.py](#) (batched VMAS), [agents/harl_runner.py](#) (train entry + monkey-patches), [agents/happo_policy.py](#) (checkpoint → VMAS policy), [agents/baselines.py](#), [agents/wildfire_task.py](#) (BenchMARL Task for MAPPO/IPPO).

```
# Smoke training – verify the pipeline runs (~3-6 s each)
python scripts/train_mappo_smoke.py      # MAPPO – centralized critic (BenchMARL)
python scripts/train_ippo_smoke.py      # IPPO – fully decentralized (BenchMARL)
python scripts/train_happo_smoke.py     # HAPPO – sequential update (HARL)

# Learning from demonstration
python scripts/train_bc_happo.py        # behaviour cloning (+ --recurrent BPTT)
python scripts/train_dagger_happo.py    # DAgger

# Headline ablation: 3 algos × 4 dropouts × N seeds + Mann-Whitney U
python scripts/comms_dropout_sweep.py --seeds 5

# Does MARL beat heuristics?
python scripts/compare_baselines.py --seeds 5 --steps 250

# TensorBoard (HARL writes events automatically)
tensorboard --logdir "results/harl_runs" --port 6006
```

Install HARL once (one-time): `cd .. && git clone https://github.com/PKU-MARL/HARL.git && cd HARL && pip install -e .`

5. Simulation model

Deep dive: [docs/simulation_overview.md](#) (conceptual) and [docs/simulation_pipeline.md](#) (end-to-end run guide)

The world is a **2D continuous plane** with a **discrete grid overlay** (default 16×16, up to 128×128). Agent motion is continuous 2D; fire, smoke, terrain type and elevation live on the grid.

5.1 Agents

Agent	Count	Max speed	Notes
Drones (aerial searchers)	3	0.5 sim/step	"2.5D" — 2D motion + tracked AGL altitude (30/60/90 m); camera detection
Ground robots (UGVs)	2	0.2 sim/step	Range-based confirmation (within ~12 m); terrain-limited traversal

5.2 Terrain

Loaded from a pre-built `.npz` cache combining **USGS 3DEP** (10 m DEM), **OpenStreetMap** (roads/water/buildings), and optionally **LANDFIRE** (fuel). Default area: **Malibu Creek State Park, California**. Each cell is one of six land-cover classes (road / open / brush / forest / rock / water) driving fire fuel, robot cost, and robot speed.

5.3 Fire & smoke

- **Fire:** stochastic cellular automaton on the grid (every 3 env steps). Ignition probability combines fire exposure, fuel, moisture, wind alignment, uphill slope, stochastic variability, and a target-area boost. Burn lifetime 5–14 update steps; burned cells are permanent.
- **Smoke:** scalar field updated every step — emission, decay ($\times 0.96$), 4-neighbor diffusion, wind advection. Degrades drone detection and enables fire spotting.

5.4 Drone perception model

- **Footprint:** $\text{radius} = \text{altitude_AGL} \times \tan(\text{FOV}/2)$ (~32 m at 30 m AGL, ~96 m at 90 m).
- **Detection probability** = product of four independent factors: distance factor, land-cover (occlusion), smoke/fire (Beer-Lambert attenuation + glare + heat shimmer), and altitude quality. This is an *abstract* stand-in for what the real YOLOv8 pipeline (Section 3) computes.

5.5 Communication & episode

- **Comms dropout:** i.i.d. Bernoulli mask per step on each agent's observation of each neighbour's relative position ($\text{observed} = \text{actual} \times \text{Bernoulli}(1-\text{rate})$).
- **Scout→Confirm protocol:** a drone scouts (stochastic camera detection), then a ground robot confirms (distance threshold ~12 m). Episode ends when all 5 survivors confirmed or step budget elapses.
- **Reward:** shared team reward (+1.0 per confirmed survivor, $-0.001/\text{step}$) plus individual credit (drone scout +0.3, ground confirm +0.5, burning-cell -1.0 , travel cost, altitude change) to drive role specialization.

*The simulator is well-suited for **comparing coordination strategies and testing robustness to comms dropout**. Fire physics, perception, and traversal are conceptually grounded but **not calibrated to field data** — a policy trained here would need significant fine-tuning before deployment.*

5.6 Pipeline

```
[1] Build terrain cache → data/terrain_cache/<name>_<grid>.npz
[2] Export trajectories → web/trajectories/<strategy>.json
[3] View in browser    → http://localhost:8080
[4] EDA reports (opt.) → results/eda/*.html
```

```
# Build terrain (optionally with LANDFIRE fuel data)
python scripts/build_real_terrain_cache.py \
  --place "Malibu Creek State Park, California" --grid-size 128 \
  --landfire-email your@email.com
```

```
# Export baseline + HAPPO trajectories for the viewer
python scripts/export_trajectories.py
```



```
# Serve the React + Three.js viewer
python -m http.server -d web 8080 # → http://localhost:8080
```

6. Datasets & data sources (links)

6.1 Computer vision imagery

Source	Use in project	Real/sim	Link
SARD (Search-And-Rescue UAV dataset)	Real drone-view survivor cutouts, GrabCut-segmented to <code>data/cv_assets/sard_grabcut/</code> (54 reviewed)	Real	Kaggle: sambolek/sard-search-and-rescue
NII-CU MAPD (multispectral aerial person detection)	Demo survivor assets from the public annotated sample	Real	https://www.nii-cu-multispectral.org/
NAIP (USDA aerial ortho-imagery)	Real terrain background for CV crops; fetched per bbox by <code>detection/naip.py</code>	Real	https://imagery.nationalmap.gov/arcgis/rest/services/USGSNAIPPlus/ImageServer
Procedural color-field noise	Fallback training background (the "procedural trap")	Sim	— (generated in-repo)
Wildfire effects (flames/smoke/burn)	Rendered overlay on every CV frame	Sim	<code>detection/wildfire_effects.py</code>

6.2 Terrain / geospatial

Source	Use	Link
USGS 3DEP	10 m digital elevation model (elevation, slope)	https://elevation.nationalmap.gov/arcgis/rest/services/
OpenStreetMap	Roads, water bodies, buildings, land cover	https://www.openstreetmap.org
LANDFIRE	Vegetation fuel / canopy / fire-behaviour fuel models (optional, requires free account email)	https://lfps.usgs.gov/api

6.3 Referenced but not yet integrated

- **NIFC** (National Interagency Fire Center) — real wildfire data (project plan §17.7).

6.4 Code that pulls these sources

`terrain/usgs_osm_builder.py` · `terrain/landfire_client.py` · `terrain/real_terrain.py` · `detection/naip.py` · `scripts/extract_sard_assets.py` · `scripts/extract_nii_cu_sample_assets.py` · `scripts/build_real_terrain_cache.py`

Note on SARD: the code references a local `--sard-root` checkout. The Kaggle link above is the canonical public origin of that dataset (Sambolek & Ivasic-Kos). All other links are taken directly from the source files.

7. Mission metrics

Every coordination strategy — hand-coded baseline or trained policy — is scored on the same six metrics. Implementation: [evaluation/mission_metrics.py](#).

Metric	Direction	Definition
survivor_recall	higher	fraction of survivors confirmed by a ground robot
time_to_verification	lower	avg steps between drone scout and ground confirmation
false_positive_trips	lower	ground-robot trips to a location with no survivor
hazard_exposure	lower	step-count ground robots spend on burning cells
ugv_travel_cost	lower	total ground-robot path length
drr (across dropouts)	higher	degradation resilience ratio under comms loss

Hand-coded baselines ([agents/baselines.py](#)): [random](#), [lawnmower](#), [nearest_candidate](#), [highest_confidence](#).

8. Repository map

```
omnisearch/
├── envs/wildfire_search.py      # WildfireSearchScenario (VMAS)
├── agents/
│   ├── wildfire_task.py        # BenchMARL Task wiring (MAPPO / IPP0)
│   ├── train_helpers.py        # smoke_config() / research_config()
│   ├── baselines.py            # Hand-coded coordination strategies
│   ├── harl_env.py             # HARL-shape adapter (single env) for HAPPO
│   ├── harl_vec_env.py         # Batched VMAS vec env for HAPPO
│   ├── harl_runner.py          # train_happo() entry point + monkey-patches
│   └── happo_policy.py         # Load HAPPO checkpoint → VMAS policy
├── detection/
│   ├── fire_detector.py        # Fire → YOLOv8 person pipeline (real images)
│   ├── person_detector.py      # HSV thresholding + connected components
│   ├── naip.py                 # YOLOv8 wrapper, classes=[0]
│   ├── wildfire_effects.py     # NAIP aerial imagery tile cache
│   └── pipeline.py             # Flame/smoke/burn overlays
├── terrain/                    # Two-stage: fire-first, then person
├── evaluation/                 # USGS / OSM / LANDFIRE terrain builders
│   ├── mission_metrics.py      # The 6 metrics + DRR
│   ├── closed_loop.py          # Sim rollout + per-frame detection + GT scoring
│   ├── sim_renderer.py         # Synthetic UAV top-down view
│   └── trajectory_export.py     # Per-step state → JSON for the web viewer
├── scripts/                   # train_*, sweeps, terrain build, extract_*, eda
├── notebooks/                 # 01 setup · 02 detection · 03 sweep · 04 closed-loop · 05 baselines
├── results/                   # Training artifacts (gitignored)
└── web/                       # React + Three.js strategy viewer (index.html / index2d.html)
```

9. Source documents

This file consolidates the following standalone docs — consult them for the fullest detail:

- [docs/CV_SURVIVOR_DETECTION.md](#) — computer-vision pipeline, results, domain gaps.

- [docs/RL_TRAINING_EXPERIMENTS.md](#) (+ [.pdf](#)) — MARL/HAPPO training log & conclusions.
- [docs/simulation_overview.md](#) — conceptual model of the simulated world and its simplifications.
- [docs/simulation_pipeline.md](#) — end-to-end build → export → view → EDA guide.
- [docs/ISAAC_LAB_COSMOS3_QUICKSTART.md](#) / [docs/ISAAC_REMOTE_GPU_SETUP.md](#) — high-fidelity Isaac Lab path.
- [README.md](#) — setup, run instructions, project status.
- [notebooks/README.md](#) · [web/README.md](#) — component-level walkthroughs.